

Practice Question

Q1. Compare Bag-of-Words, Normalized Term Frequency, and TF-IDF ?

Ans->

- **Program:-**

```
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

corpus = [
    "Natural language processing enables computers to understand human language",
    "Word embeddings like Word2Vec represent words in vector space",
    "Bag of words and TF-IDF are traditional NLP approaches",
    "Deep learning models improve performance in NLP tasks"
]

# Bag-of-Words
count_vectorizer = CountVectorizer()
count_matrix = count_vectorizer.fit_transform(corpus)

bow_df = pd.DataFrame(count_matrix.toarray(),
    columns=count_vectorizer.get_feature_names_out())

# Normalized Term Frequency
norm_tf = bow_df.div(bow_df.sum(axis=1), axis=0)

# TF-IDF
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(corpus)

tfidf_df = pd.DataFrame(tfidf_matrix.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out())

print("Bag-of-Words:\n", bow_df)
```

```
print("\nNormalized Term Frequency:\n", norm_tf)

print("\nTF-IDF:\n", tfidf_df)
```

• Output:-

```
PS D:\NLP\Exp5> python -u "d:\NLP\Exp5\Practice5_1.py"
Bag-of-Words:
  and approaches are bag computers ... understand vector word word2vec words
0  0          0  0  0      1 ...           1      0      0      0      0
1  0          0  0  0      0 ...           0      1      1      1      1
2  1          1  1  1      0 ...           0      0      0      0      1
3  0          0  0  0      0 ...           0      0      0      0      0

[4 rows x 32 columns]

Normalized Term Frequency:
  and approaches are bag computers ... understand vector word word2vec words
0  0.0          0.0 0.0 0.0  0.111111 ...  0.111111 0.000000 0.000000 0.000000 0.000000
1  0.0          0.0 0.0 0.0  0.000000 ...  0.000000 0.111111 0.111111 0.111111 0.111111
2  0.1          0.1 0.1 0.1  0.000000 ...  0.000000 0.000000 0.000000 0.000000 0.100000
3  0.0          0.0 0.0 0.0  0.000000 ...  0.000000 0.000000 0.000000 0.000000 0.000000

[4 rows x 32 columns]

TF-IDF:
  and approaches are bag computers ... vector word word2vec words
0  0.000000  0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000
1  0.000000  0.000000 0.000000 0.000000 ... 0.348299 0.348299 0.348299 0.274603
2  0.328919  0.328919 0.328919 0.328919 ... 0.000000 0.000000 0.000000 0.259324
3  0.000000  0.000000 0.000000 0.000000 ... 0.000000 0.000000 0.000000 0.000000

[4 rows x 32 columns]
```

Q2. Explore Semantic Relationships with Word2Vec

- Train a Word2Vec model on a given text dataset (news articles, product reviews, or Wikipedia sample).
- Find vector embeddings for 5 chosen words (e.g., “king”, “queen”, “man”, “woman”, “computer”).
- Use the model to find most similar words to each chosen word.
- Perform a vector arithmetic test
- Present the results in tabular or graphical format.

- **Program:-**

```
import pandas as pd
import matplotlib.pyplot as plt
from gensim.models import Word2Vec
# Sample corpus (replace with your dataset)
corpus = [
    "The king and queen ruled the kingdom.",
    "A man and a woman are walking in the park.",
    "The computer is an amazing invention.",
    "Queen Elizabeth is a famous monarch.",
    "A programmer uses a computer to solve problems.",
    "The king greeted the people.",
    "Women and men have equal rights.",
    "The queen and the king attended the ceremony."
]
# Tokenize sentences
tokenized_corpus = [sentence.lower().split() for sentence in corpus]
# Train Word2Vec model
model = Word2Vec(sentences=tokenized_corpus, vector_size=100, window=5, min_count=1,
workers=2, seed=42)
# Chosen words
words = ["king", "queen", "man", "woman", "computer"]
# 1. Vector embeddings
embeddings = {word: model.wv[word] for word in words}
# 2. Most similar words
similar_words = {word: model.wv.most_similar(word, topn=5) for word in words}
```

3. Vector arithmetic: king - man + woman $\approx$?

```
result_word, similarity = model.wv.most_similar(positive=['king', 'woman'], negative=['man'])[0]
```

4. Present results

```
print("Word Embeddings (first 5 values):")
```

```
for word in words:
```

```
    print(f'{word}: {embeddings[word][:5]}')
```

```
print("\nMost Similar Words:")
```

```
for word in words:
```

```
    print(f'{word}: {[w for w, s in similar_words[word]]}')
```

```
print(f'\nVector Arithmetic (king - man + woman): {result_word} (similarity: {similarity:.4f})')
```

Tabular presentation

```
sim_df = pd.DataFrame({word: [w for w, _ in similar_words[word]] for word in words})
```

```
print("\nMost Similar Words Table:\n", sim_df)
```

Optional: Graphical presentation (bar chart for similarity scores of 'king')

```
king_sim = similar_words["king"]
```

```
plt.figure(figsize=(8,4))
```

```
plt.bar([w for w, _ in king_sim], [s for _, s in king_sim], color='skyblue')
```

```
plt.title("Top 5 Similar Words to 'king'")
```

```
plt.ylabel("Similarity Score")
```

```
plt.show()
```

- **Output:-**

```
PS D:\NLP\Exp5> python -u "d:\NLP\Exp5\Practice5_2.py"
Word Embeddings (first 5 values):
king: [-0.00272333  0.00652657 -0.0097982  0.00792107  0.00085544]
queen: [ 9.9604521e-03  5.5450858e-03 -3.5894178e-03  9.4400235e-03
-1.7511577e-05]
man: [ 0.0061672  0.00473556  0.00116949  0.00773203 -0.00744674]
woman: [ 0.00758043 -0.00044988  0.00969533  0.00522865  0.00301767]
computer: [ 0.00232767  0.00557616  0.00477704 -0.00731165  0.00652067]

Most Similar Words:
king: ['man', 'problems.', 'are', 'and', 'invention.']
queen: ['problems.', 'programmer', 'king', 'invention.', 'women']
man: ['computer', 'famous', 'walking', 'a', 'king']
woman: ['solve', 'greeted', 'ceremony.', 'famous', 'uses']
computer: ['man', 'amazing', 'walking', 'rights.', 'famous']

Vector Arithmetic (king - man + woman): solve (similarity: 0.2314)

Most Similar Words Table:
      king      queen      man      woman computer
0      man  problems.  computer      solve      man
1  problems.  programmer    famous  greeted  amazing
2      are      king  walking  ceremony.  walking
3      and  invention.      a    famous  rights.
4  invention.      women    king      uses    famous
```

Figure 1

